Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O que o enunciado pede e o que você entrega. Não explique a implementação aqui.

O enunciado pede sincronizar passageiros e carros de metrô, respeitando as vagas e aguardando a confirmação dos embarques. Entrego a implementação em C/Pthreads, Makefile e testes. O teste do professor atendeu 100 passageiros. A bateria adicional aprovou 12 cenários, com carga de até 1.024 passageiros simultâneos e 300 rodadas aleatórias.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Implementação em C/Pthreads no Linux	OK	GCC no Ubuntu/WSL; log.
Um mutex por estação, sem semáforos	OK	sync_car.h e sync_car.c.
Aguardar vagas sem espera ocupada	OK	pthread_cond_wait; teste_espera.
Respeitar o limite de assentos	OK	Professor: 100 passageiros.
Reservar antes das confirmações	OK	teste_limites: reservas em grupo.
Esperar a última confirmação	OK	teste_limites: cenários 6 e 7.
Tratar estação vazia e carro sem vagas	OK	teste_limites: cenários 1 e 2.
Makefile para gerar sync_car e limpar	OK	Makefile; conferência com make -Bn.

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
No Ubuntu/WSL, na pasta do projeto:
make clean
make
```

Executar (exemplo):

```
./sync_car
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. sync_car.c - inicialização, reserva de vagas e confirmação do embarque.
2. sync_car.h - struct station, um mutex, duas condições e três contadores.
3. sync_car-runner.c - teste original do professor com 100 passageiros.
4. teste_init.c e teste_espera.c - testes parciais, preservados no GitHub.
5. teste_limites.c - 12 cenários de estresse, preservados no GitHub.
6. Makefile e evidencias.log - compilação e registros incluídos na entrega.

Decisões técnicas:

1. Um mutex protege esperando, vagas e embarcando em cada estação.
2. As condições suspendem passageiros e carro. O while verifica novamente o estado após cada aviso.
3. A reserva aumenta embarcando; station_on_board confirma o assento. O carro só retorna quando não há confirmações pendentes e está cheio ou não há mais passageiros esperando. As vagas restantes são zeradas antes da partida.

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Mutex e condições	Contadores de espera, vagas e embarques pendentes.	Sem troca; implementação construída em etapas.
-	Não utilizada	A abordagem com mutex e condições foi mantida.	-
-	Não utilizada	Os testes ampliaram a validação, sem mudar a abordagem.	-

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Inicializar a estação e testar.	OK	-	02/11 18:09	Log: inicialização PASS.
2	S1	Repetir o teste no terminal.	OK	-	02/11 18:15-18:27	Log: testar-init PASS.
3	S1	Acrescentar a espera do passageiro.	Falha	Função station_wait_for_car duplicada.	03/11 17:46:44	Log: redefinition; saída 2.
4	S1	Remover a duplicação e recompilar.	OK	-	03/11 17:46:57	Log: init e espera PASS; saída 0.
5	S1	Repetir o teste de espera.	OK	-	03/11 17:52-17:53	Log: 6 passageiros; reservas corretas.
6	S1	Traduzir variáveis e repetir os testes.	OK	-	04/11 16:05	Log: init e espera PASS; Figura 1.
7	S1	Testar as quatro funções com o código do professor.	OK	-	04/11 16:46	Figura 2; log: 100 passageiros; Looks good!
8	S1	Executar a bateria de estresse com 300 rodadas.	OK	-	04/11 a partir de 16:55	Figura 3; log: 12/12 cenários; carga até 1.024.

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do evidencias.log com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com date, whoami e pwd, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

O log registra a duplicação de station_wait_for_car em 03/11, corrigida antes dos testes. Os testes parciais passaram e o teste do professor terminou com Looks good! para 100 passageiros. O estresse aprovou 12/12 cenários, 300 rodadas e carga até 1.024 passageiros simultâneos. Os testes estão disponíveis no GitHub. Commits publicados:

`7d7bbf6` `a81346c` `277fb14`

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.

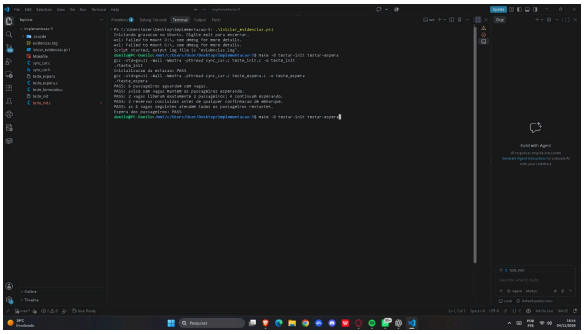


Imagem 1 - Testes parciais gerados com IA.

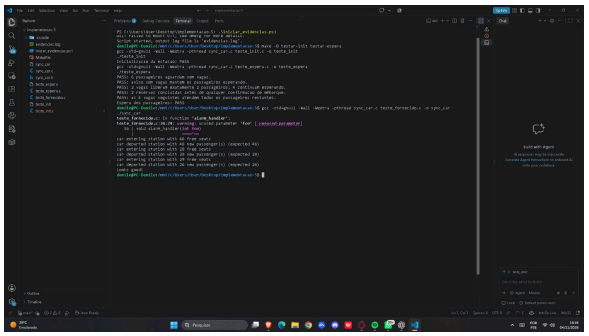


Imagem 2 - Teste fornecido pelo professor: Looks good!

- Validação final: teste passando / saída correta.

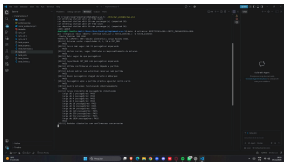


Imagem 3 - Testes de estresse que pedi à IA para gerar.

Prints extras só se algo excepcional aconteceu e o log sozinho não explica, você precisa ter esse feeling, mas posso dar uma ajuda (ex.: comportamento visual, crash com stack trace longo).

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA (2–3 linhas):

Usei IA para apoiar a implementação e criar testes parciais e de estresse, executados no Ubuntu/WSL. Os prompts seguem na próxima página. Os arquivos dos testes estão disponíveis no GitHub.

Uso de IA: prompts e resultados dos testes - 04/11/2026

Prompts enviados: "faça um teste pra isso".
"baseado nesses testes, faça um arquivo de testes mais complexos e que vao testar o programa até o limite dele, preciso da garantia que meu programa vais passar nos testes".
"nao execute, mande o resultado esperado e o comando pra eu rodar".

"organize o relatorio e adicione essas imagens na area de evidencias, coloque a descriçao, que na imagem 1 mostra o resultado do teste feito por ia, na 2 a que o professor passou e a 3 o teste final gerado por ia".

Os testes e os comandos adicionais referem-se à versão 277fb14, disponível no GitHub.
As execuções no Ubuntu/WSL estão registradas no evidencias.log.

Resultado registrado: professor, 100 passageiros e Looks good!; estresse, 12/12 cenários, 300 rodadas, 24.881 passageiros e 809 carros. Carga simultânea até 1.024; sem falhas ou inconclusivos.

```
make -B testar-init testar-espera
make -B estressar REPETICOES=300 LIMITE_PASSAGEIROS=1024
```

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

Separar a reserva do assento da confirmação do embarque foi a decisão central.
O mutex protege os contadores e as condições suspendem as threads sem espera ocupada.
A duplicação de uma função causou falha de compilação e foi removida antes dos testes.
Os testes verificaram a última confirmação, as vagas limitadas e a carga simultânea.
Esses resultados aumentam a confiança, mas não provam correção em todo escalonamento.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	GCC: sim; Makefile: conferido.
Rodei os testes e evidencias.log foi gerado	Sim; evidencias.log no GitHub.
Tenho prints obrigatórios	3 prints; erro registrado no log.
Testei pelo menos um caso limite e um caso inválido	Limites: sim; inválidos: não.
Preenchi a seção de uso de IA	Sim; prompts e arquivos indicados.
Revisei o relatório e removi frases genéricas / vazias	Sim.

10. Se eu tivesse mais 2 horas

Eu repetiria a bateria com outras sementes e testaria a solução em outra instalação Linux. Também usaria ferramentas para detectar corridas de dados e verificaria a distribuição das vagas entre os passageiros. Assentos negativos e chamadas fora de ordem não foram testados; a validação realizada segue o fluxo do enunciado. Essas verificações adicionais ficam como melhoria.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.